

C Programming

5_arrays

Average score?

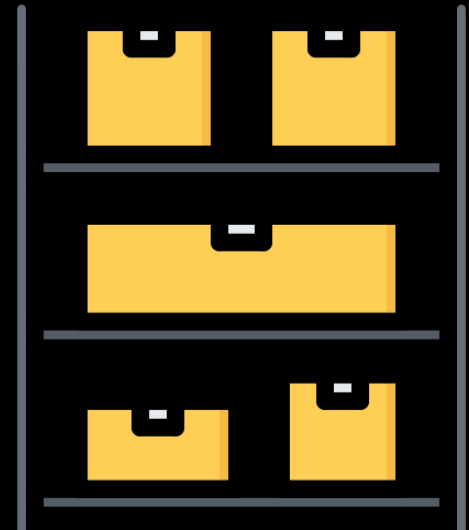
	score1	score2	score3	
	77	43	100	
	Паша	Маша	Даша	

Arrays

Массивы используются для хранения нескольких значений в одной переменной вместо объявления отдельных переменных для каждого значения.

Если переменная - это коробка со значением, то массив - это шкаф с этими коробками

Массивы во многих ЯП (в том числе Си) умеют хранить значения **только одного** типа данных, т.е. один массив не может хранить сразу и **целые** и **вещественные** типы данных



Arrays

```
int scores[3] = {77, 43, 100};
```

	[0]	[1]	[2]	
Value	77	43	100	
Address	200	204	208	

Массивы в C/C++, Python, Java, Go и многих других ЯП индексируются с 0
Исключения, где массивы начинаются с индекса 1: MATLAB, lua, R, Julia..

```
printf("Scores: ");  
for (int i = 0; i < 3; i++) {  
    printf("%d, ", scores[i]);  
}  
printf("\n");
```

```
#define N 5
```

```
int a[N];
```

```
int b[5] = {1, 2, 3, 4, 5};
```

```
float c[] = {1.5, 3.2, 2.5};
```

```
int d[6] = {[3] = 9};
```

```
char e[N] = {0};
```

```
int a[5] = {5, 6, 7, 8, 9};
```

```
int b[5] = {1, 2, 3, 4, 5};
```

```
b = a; // Error
```

```
a = {5, 3}; // Error
```

```
a[2] = -17; // Ok
```

```
a[0] = b[4]; // Ok
```

```
a[2.5] = 33; // Error
```

```
b[4000] = 100; // Error, segfault
```

```
a[-1] = 8; // Ok, possible segfault
```

Segmentation fault - ошибка во время выполнения программы, когда программа попыталась обратиться к памяти, к которой доступ не разрешен.

array size

```
int m[] = {1, 2, 3}; // 3 элемента  
int array_size = sizeof(m) / sizeof(m[0]); // 12/4=3
```

Delete/insert

В "обычном" массиве нельзя удалить элемент и уменьшить/увеличить размер самого массива. Такое можно будет делать с **динамическими массивами**, скоро их посмотрим

```
void print_arr(int n, int array[n]) {  
    for (int i = 0; i < n; i++) {  
        printf("array[%d] = %d\n", i, array[i]);  
    }  
    printf("\n");  
}
```

```
int scores[N] = {77, 43, 100, 55, 11};  
print_arr(N, scores);
```

```
int array[M] = {1, 2, 3};  
print_arr(M, array);
```

Random numbers

```
#include <stdlib.h>  
// ...  
int random = rand();
```

rand() - ф-ия, что возвращает случайное **целое** число (целое в диапазоне от 0 до RAND_MAX, где RAND_MAX - определен в stdlib.h)

Random numbers

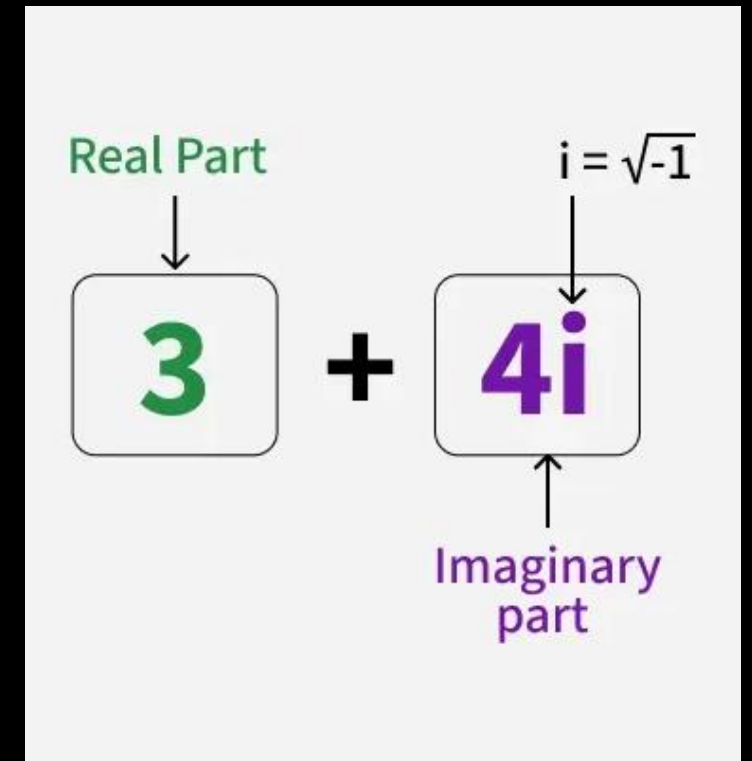
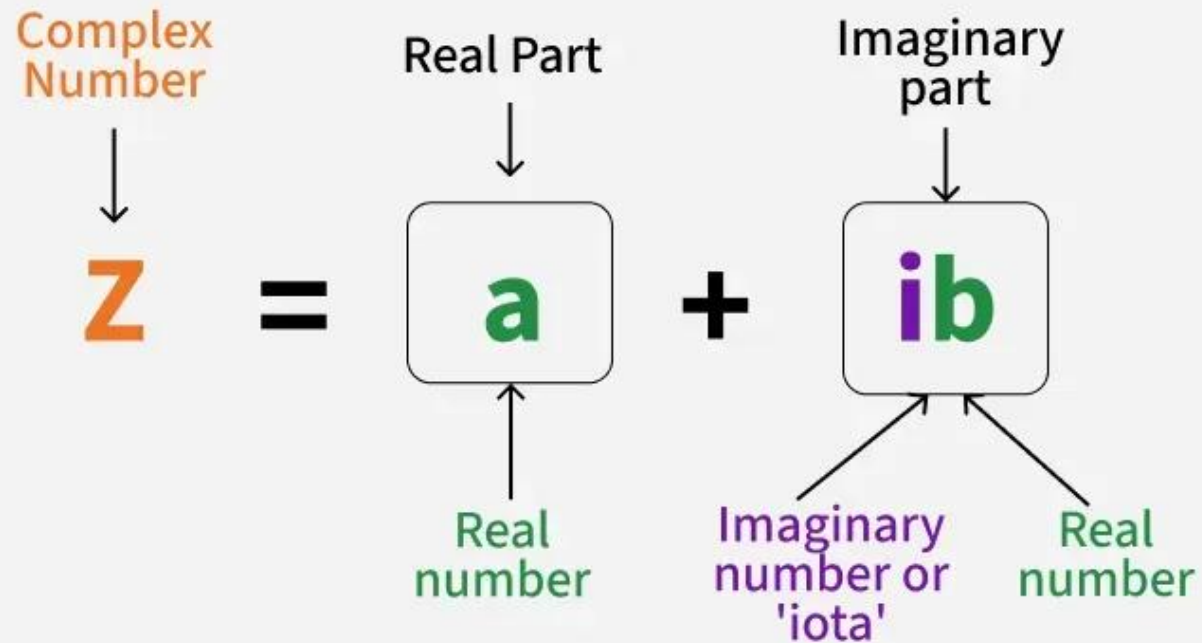
- **srand()** - ф-ия задания seed (семя) для псевдорандома. Seed - некоторое число, что каким-то образом влияет на формулу случайных чисел
- **time()** - ф-ия возвращающая кол-во секунд прошедших с 00:00:00 UTC, 1 Января, 1970 года, называется - Unix timestamp (timestamp - временная метка).

<https://www.unixtimestamp.com/>

```
srand(time(NULL)); // достаточно сделать 1 раз  
printf("Unix timestamp = %ld\n", time(NULL));  
printf("Random [0..99] = %d\n", rand() % 100);
```

Самостоятельно: заполнить массив размером N случайными числами от 0 до 999 и найти в нем минимальный элемент - вывести его на экран

Complex Numbers



Комплексное число – выражение вида $a + bi$, где a и b некоторые целые либо вещественные числа, а i это корень из -1 .

Как представить комплексное число и положить его в компьютер?

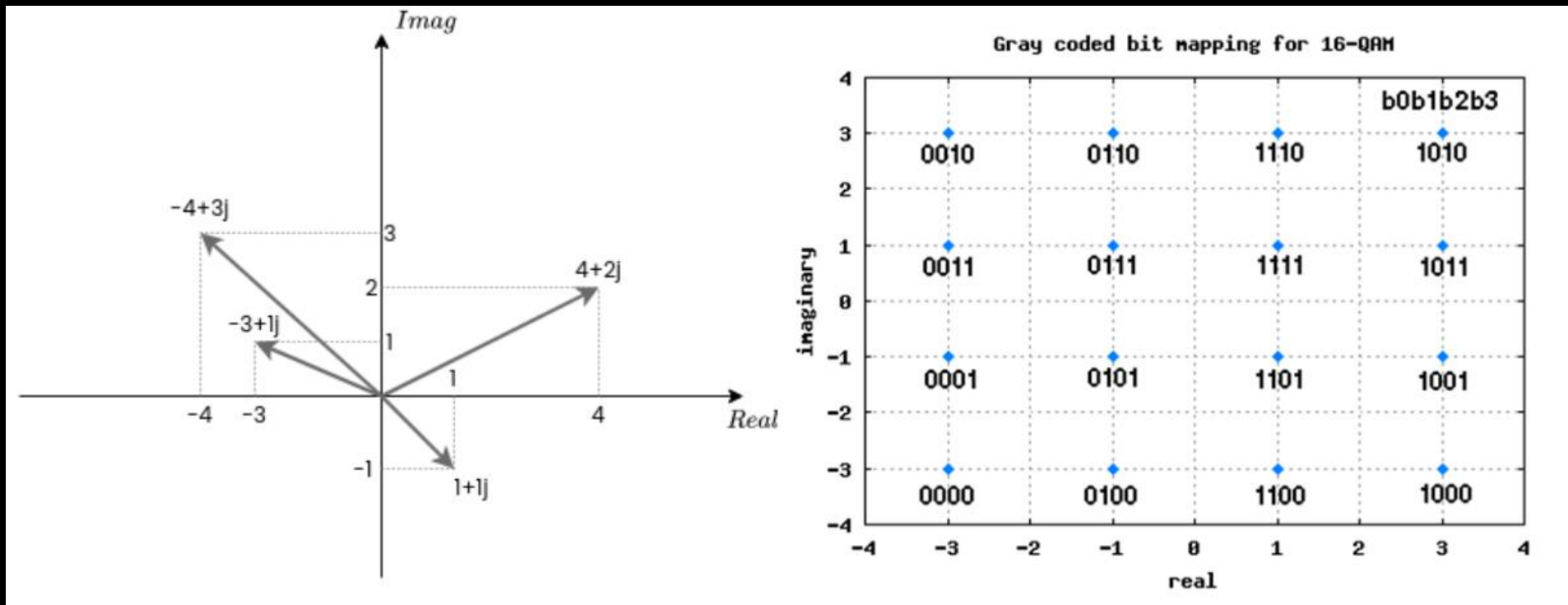
Одно **комплексное число** создают из двух чисел — a и b , а i пропускается. Соответственно массив из 3 комплексных чисел есть массив из 6 обычных чисел. И тогда числа под четными индексами — это **реальная часть** комплексного числа, а нечетные индексы **мнимая часть**:

[RE0, IM0, RE1, IM1, RE2, IM2],

- RE — Real (реальная часть),
- IM — Imag (imaginary, мнимая часть).

Так, например, наши с вами телефоны из воздуха достают большие массивы **комплексных чисел**, когда работают с беспроводными сетями типа **Wi-Fi** или **4G**, **5G**..

Эти комплексные числа образуют различные "**созвездия**", которые затем преобразуются в более понятные анные из `0` и `1`



Допустим нам из воздуха пришло 10 комплексных целых чисел, где реальная и мнимая части занимают по 16 бит, как мы можем упаковать их в массив?

```
short int complex[20];
```

- `complex[четный_индекс]` - реальная часть
- `complex[нечетный_индекс]` - мнимая часть

```
int complex[10];
```

- `complex[i]` - комплексное число
 - `(complex[i] & 0xffff)` - реальная часть
 - `(complex[i] >> 16) & 0xffff` - мнимая часть

Сортировки - алгоритмы для упорядочивания данных в массивах (по возрастанию или убыванию).

Алгоритмов сортировок существует очень много и с разными данными они работают с разной эффективностью (**сложностью**). С отсортированными данными обычно проще и быстрее работать

Пару алгоритмов сортировок:

- Bubble Sort - Пузырьковая сортировка
- Insertion Sort - Сортировка вставками
- Selection Sort - Сортировка выбором
- Counting Sort - Сортировка подсчетом
- Radix Sort - Поразрядная сортировка
- Merge Sort - Сортировка слиянием
- Quick Sort - Быстрая сортировка

swap

```
int temp = array[0];  
array[0] = array[1];  
array[1] = temp;
```

*Имя temp или tmp - от слова временный (**temporary**)

Bubble sort

```
для i от 0 до n-1:  
    для j от 0 до n-i-1:  
        ЕСЛИ arr[j] > arr[j+1]:  
            поменять местами arr[j] и arr[j+1]
```

6 5 3 1 8 7 2 4

Bubble sort

```
void bubble_sort(int size, int arr[size]) {  
    for (int i = 0; i < size; i++) {  
        for (int j = 0; j < size - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int tmp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = tmp;  
            }  
        }  
    }  
}
```

*В функцию можно передать массив, но из функции нельзя его вернуть

Insertion Sort (вставками)

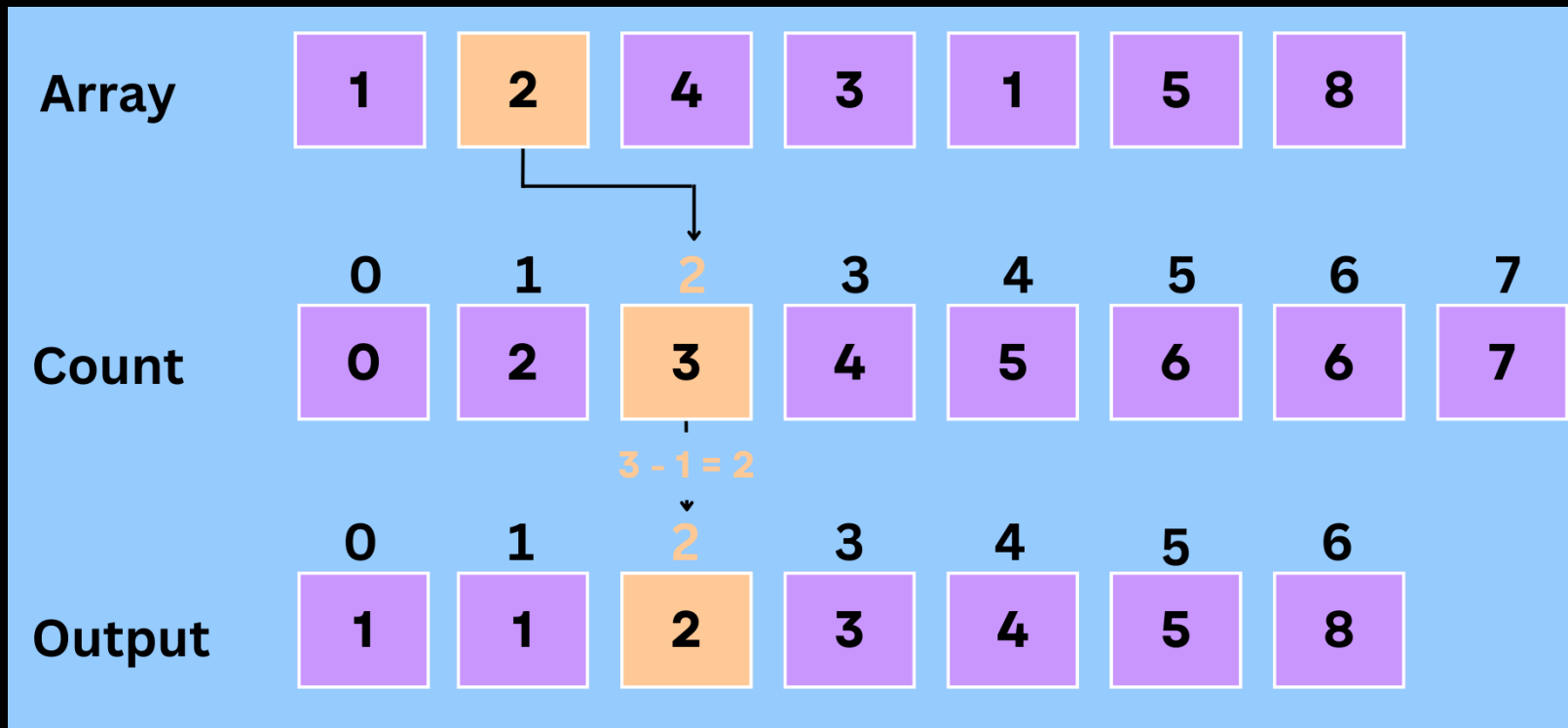
6 5 3 1 8 7 2 4

Selection sort (выбором)

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

Counting sort (Подсчетом)

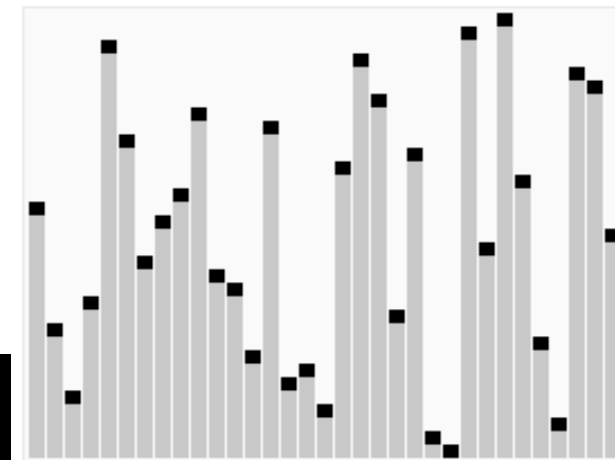
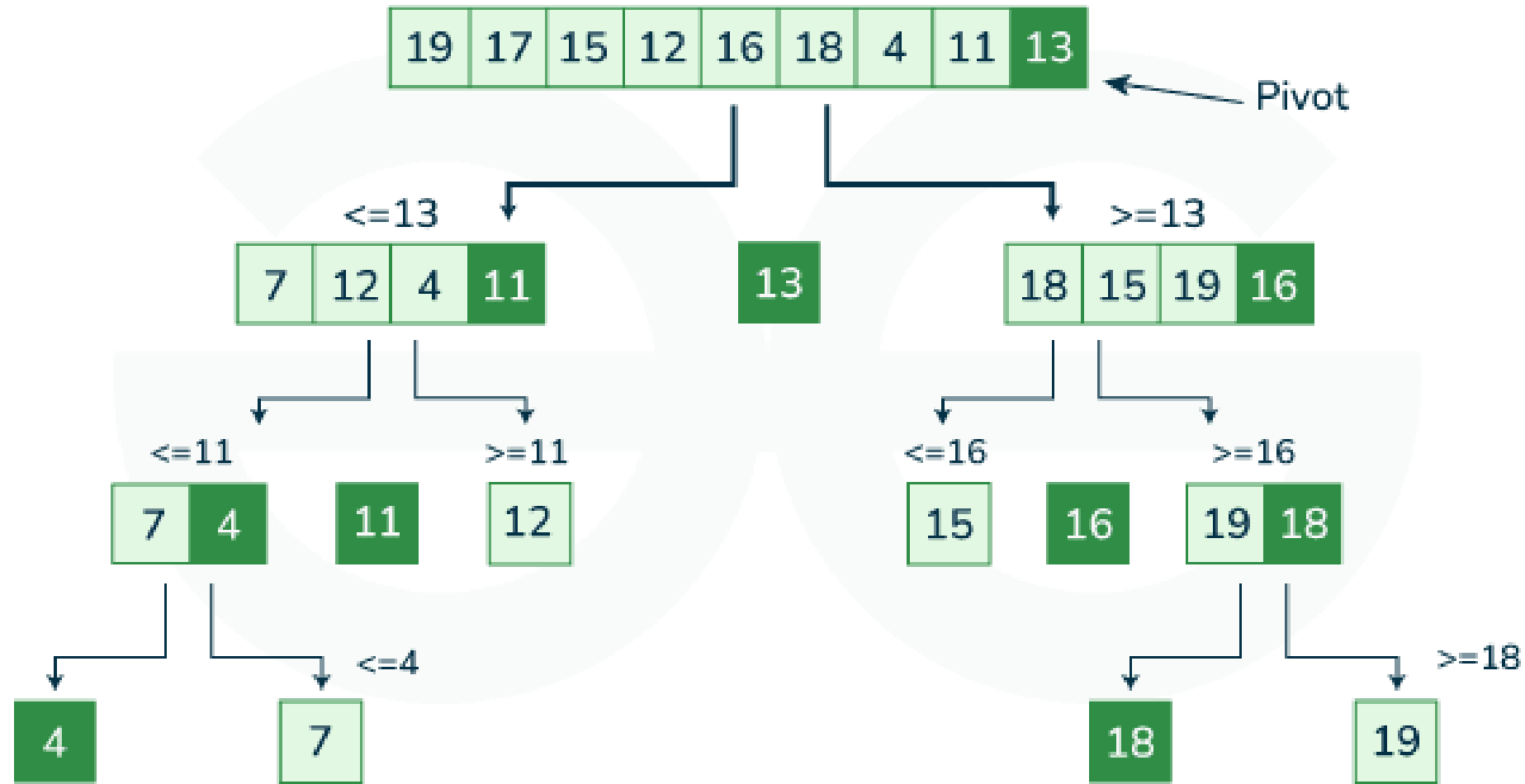
```
count[arr[i]]++;  
// i - индекс,  
// массив arr - оригинальный сортируемый массив,  
// count - массив для подсчета
```



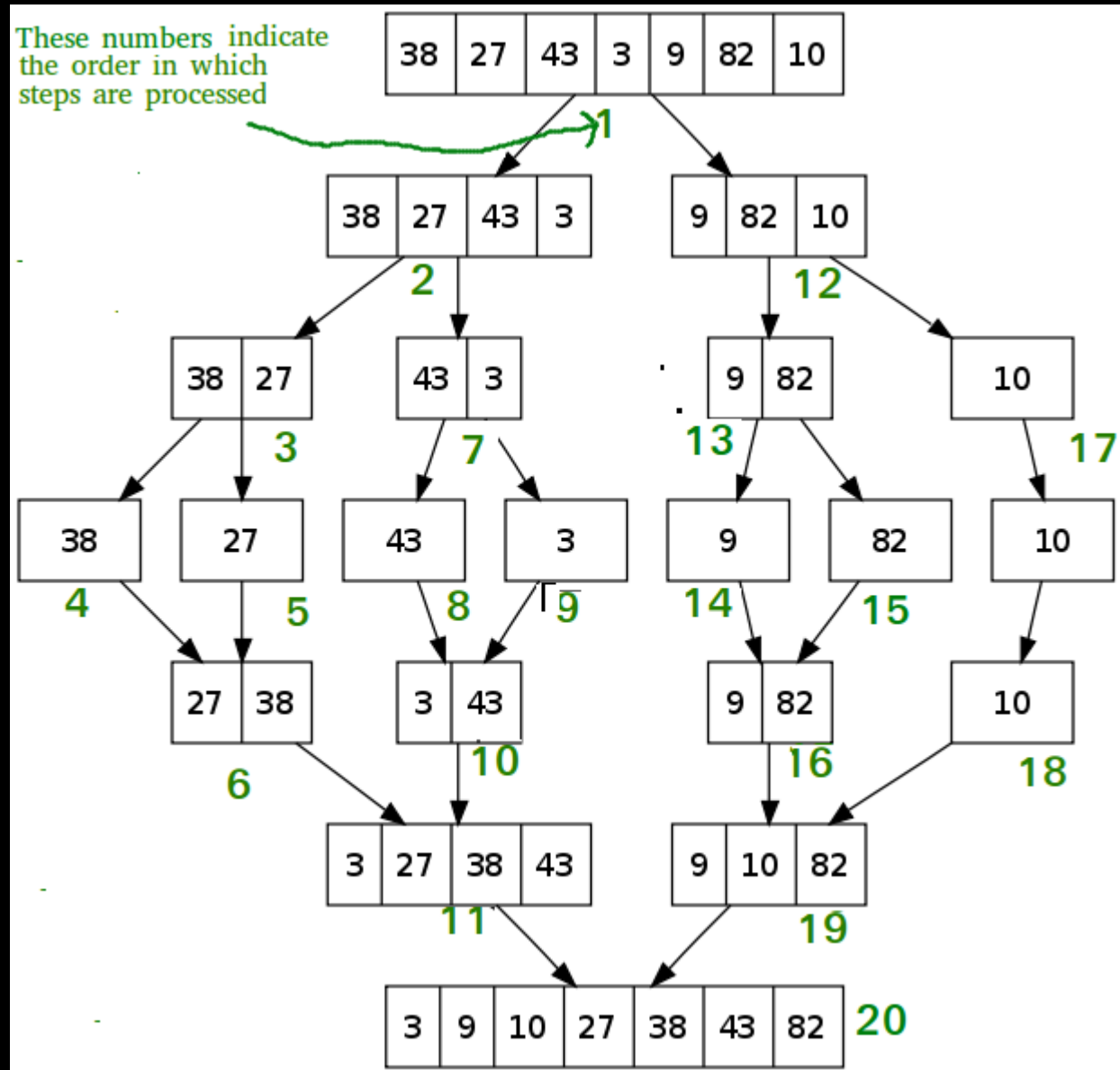
Radix sort (поразрядная)

Sort Digit 0	Sort Digit 1	Sort Digit 2	Final Result
9 5 4	4 1 1	0 0 9	0 0 9
3 5 4	9 5 4	4 1 1	3 5 4
0 0 9	3 5 4	9 5 4	4 1 1
4 1 1	0 0 9	3 5 4	9 5 4

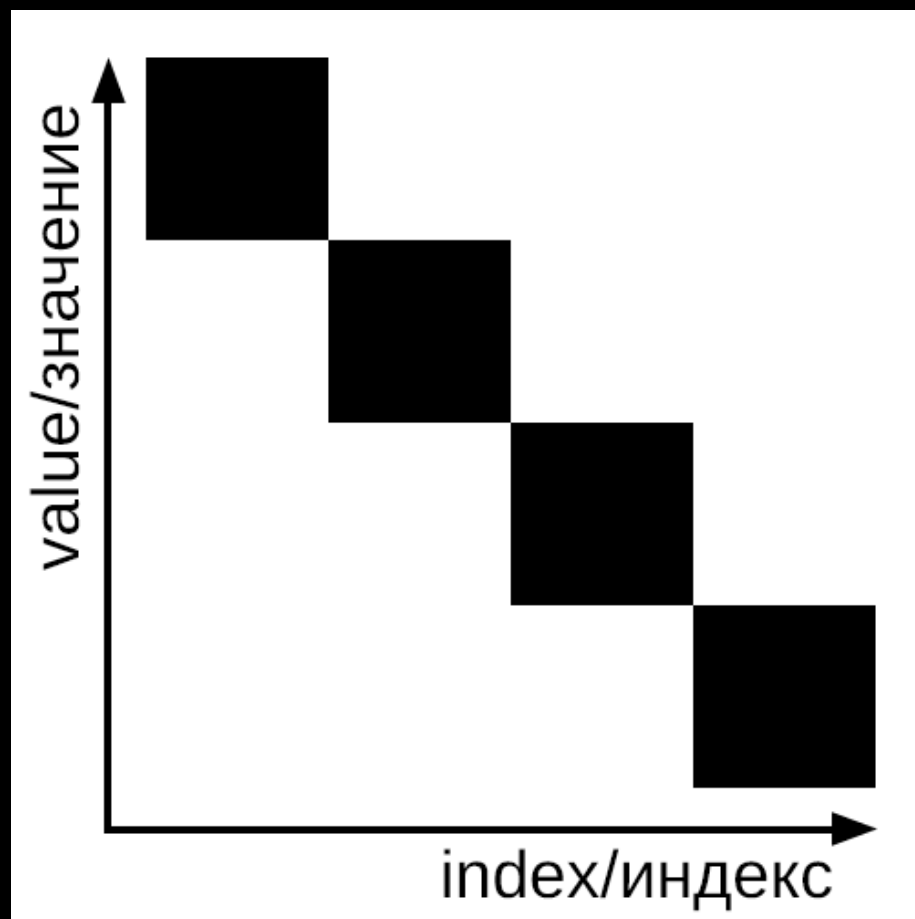
Quick sort (быстрая)



Merge sort (слиянием)



Bogo sort (непрактичная)



С такой сортировкой отсортировать массив любого размера можно с 1 раза, однако шанс не велик.. **но он есть**

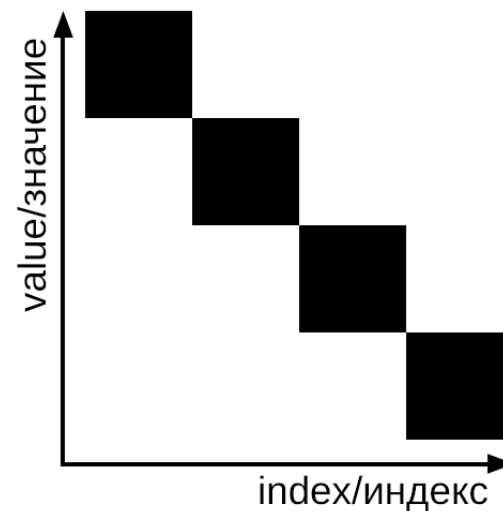
Bogo sort (непрактичная)

При прохождении цикла один раз в секунду сортировка в среднем может занять:

Кол-во элементов	Среднее время
1	1 с
2	4 с
3	18 с
4	96 с
5	10 мин
6	1,2 ч
7	9,8 ч
8	3,7 сут
9	37,8 сут
10	1,15 лет
11	13,9 лет
12	182 года

При работе 4-ядерного процессора на частоте 2,4 ГГц (9,6 млрд операций в секунду):

Кол-во элементов	Среднее время
10	0,0037 с
11	0,045 с
12	0,59 с
13	8,4 с
14	2,1 мин
15	33,6 мин
16	9,7 ч
17	7,29 сут
18	139 сут
19	7,6 лет
20	160 лет



Таким образом, колода в 32 карты будет сортироваться этим компьютером в среднем $2,7 \cdot 10^{19}$ лет.

Matrix

Переменная - коробка, массив - шкаф с коробками, матрица - комната со шкафами..

Массивы могут быть не только одномерными, могут быть двумерными.. n -мерными. Двумерные массивы обычно называют словом **матрица**, т.к. они похожи на матрицы (или таблицы) со строками и столбцами. Индексы все также начинаются с нуля!

	Column 0	Column 1	Column 2
Row 0	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
Row 1	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
Row 2	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>



```
int matrix[N][N];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        matrix[i][j] = i + j + 1;
    }
}

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
```

Memory

```
int m[3][3] = { {1, 2, 3},  
                {4, 5, 6},  
                {7, 8, 9}};
```

m[0][0] m[0][1] m[0][2] m[1][0] m[1][1] m[1][2] m[2][0] m[2][1] m[2][2]

	1	2	3	4	5	6	7	8	9	
--	----------	----------	----------	----------	----------	----------	----------	----------	----------	--

208 212 216 220 224 228 232 236 240

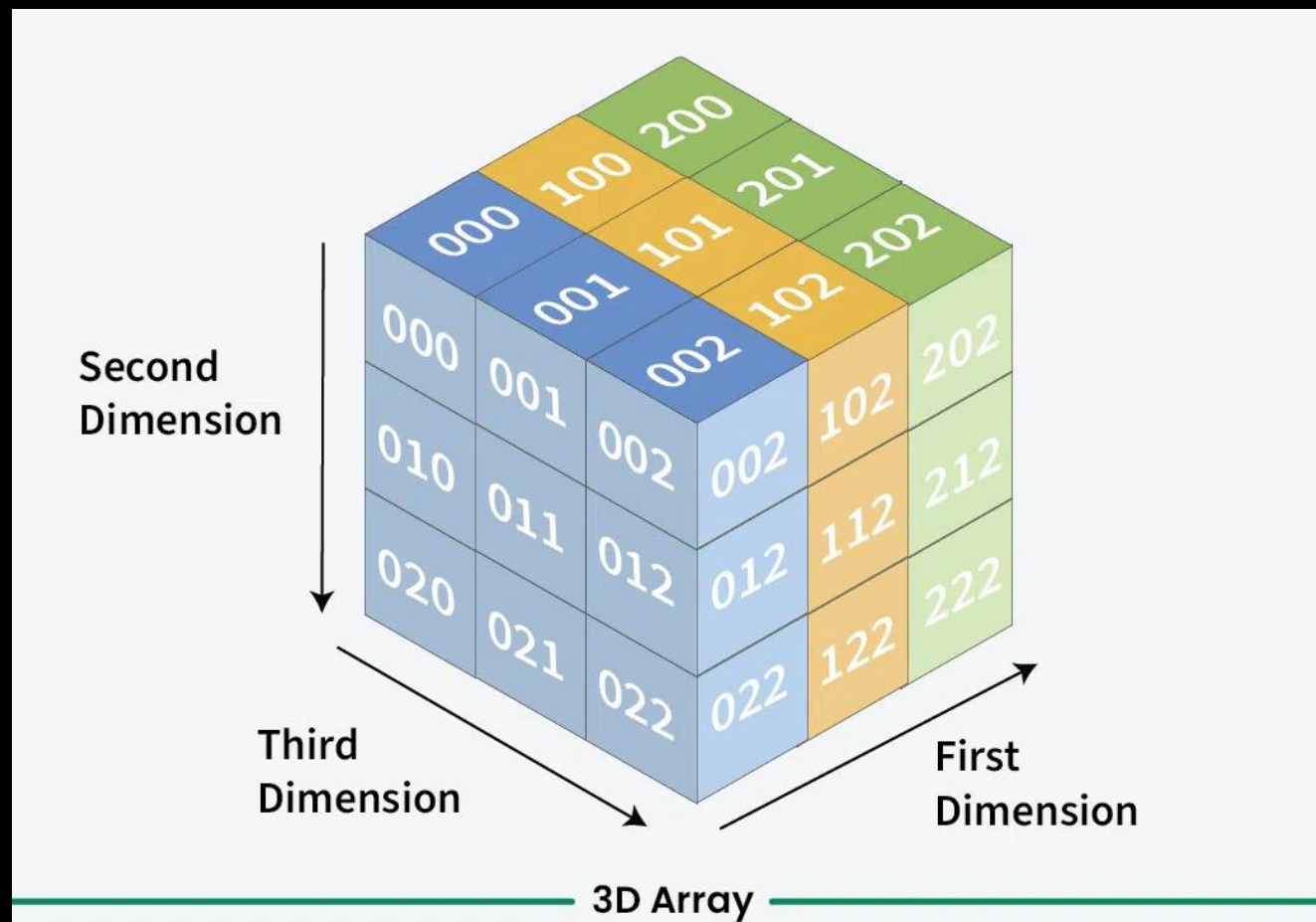
addr(m[0][2]) = addr(c[0][0]) + 0*3*sizeof(int) + 2*sizeof(int) = 208 + 0 + 2*4 = 216

1	2	3
4	5	6
7	8	9

Main Diagonal

Secondary Diagonal

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        if (i == j) {  
            printf("%d ", matrix[i][j]);  
        }  
    }  
}
```



3D массив - это уже будет квартира с комнатами со шкафами с коробками.. 4D – дом с квартирами.. и так продолжать можно долго

```
char c1 = 'H';
```

```
char c2 = 'i';
```

```
char c3 = '!';
```

```
printf("%c %c %c\n", c1, c2, c3);
```

```
printf("%d %d %d\n", c1, c2, c3);
```

String

В языке Си строка - есть массив символов.

```
char s[] = "Hi!";  
printf("%s\n", s);
```

Формат в printf - "%s" позволяет выводить строки на экран (от слова string).

Как понять где строка закончится?

NUL

s[0] s[1] s[2] s[3]

	H	i	!	\0	
--	----------	----------	----------	-----------	--

s[0] s[1] s[2] s[3]

	72	105	33	0	
--	-----------	------------	-----------	----------	--

Нул-терминатор ('\0') - это символ конца строки

String array

```
char words[2][5] = {"Hi!", "Bye!"};
```

```
printf("%s\n", words[0]);
```

```
printf("%s\n", words[1]);
```

```
printf("%c%c%c\n", words[0][0], words[0][1],  
words[0][2]);
```

```
printf("%c%c%c%c\n", words[1][0], words[1][1],  
words[1][2], words[1][3]);
```

- **scanf()** - считывает до пробела, используем формат %s, не используем амперсанд, т.к. имя массива нам даст адрес переменной массива.
- **fgets()** - считывает с пробелами

```
char name[20];  
printf("Enter name: ");  
scanf("%s", name);  
printf("Your name is %s\n", name);
```

```
printf("Enter name: ");  
fgets(name, sizeof(name), stdin);  
printf("My name is %s\n", name);
```

stdin – стандартный ввод, то куда попадают наши вводимые данные в ОС

stdout – стандартный вывод

stderr – стандартный вывод для ошибок

```
char buffer[8];  
scanf("%s", buffer);
```

```
char buffer[8];  
scanf("%7s", buffer);
```

string.h

```
#include <string.h> // Для работы с функциями
                    // обработки строк

strlen();          // Длина строки, не включая '\0'
strcpy();          // Копирование одной строки в другую
strncpy();         // Тоже самое, но не выходя за размер n
puts();           // Вывод на экран строки
fgets();           // Ввод с клавиатуры строки
strcmp();          // Сравнение двух строк
strcat();          // Объединение двух строк
```

Github

<https://github.com/kruffka/C-Programming>

